Memory-Efficient Fuzzy Convolutional-Recurrent Networks for High-Frequency Financial Time Series

Harsh Vardhan Singh
Abhishek Tiwari
Faculty of Technology
University of Delhi, India
Roll No: 23293916028, 23293916035

***Abstract*—Convolutional Neural Networks (CNNs) have emerged as powerful tools for financial time series analysis and algorithmic trading. While the pooling layer is a critical component for dimensionality reduction, traditional methods like Max Pooling (MP) and Average Pooling (AP) may not adequately capture the nuanced patterns in financial data. This study conducts a comprehensive evaluation of ten different pooling strategies - including traditional, fuzzy-based, and novel attention-based methods - for cryptocurrency price prediction. Our extended framework introduces four novel pooling layers (AAP, MSFP, TAP, HSFP) alongside established methods. Experimental results on Bitcoin data reveal that Adaptive Attention Pooling (AAP) achieves the highest performance with 84.36**

Keywords: Algorithmic Trading, Convolutional Neural Networks, Pooling Layers, Fuzzy Logic, Attention Mechanisms, Cryptocurrency Prediction, Deep Learning

***Index Terms*—Algorithmic Trading, Fuzzy Neural Networks, CNN, GRU, High-Frequency Data, Financial Time Series**

I. Introduction

Algorithmic trading systems operating on high-frequency limit order book data require models that are both expressive and computationally efficient. Financial microstructure data is characterized by extreme noise, non-linearity, and temporal dependencies. Conventional deep learning models often suffer from excessive memory consumption and slow inference, limiting their applicability in real-time environments.

To address these challenges, this work explores a structured progression of neural architectures combining convolutional learning, fuzzy logic, and recurrent modeling. The key objective is to balance predictive performance with strict memory and latency constraints.

II. Related Work

CNNs have been widely applied to financial time series by transforming temporal data into image-like representations [1]. Recurrent architectures such as LSTMs and GRUs have demonstrated effectiveness in modeling temporal dependencies [2]. Fuzzy logic has been integrated with neural networks to improve robustness under uncertainty [3]. However, few works address the combined problem of fuzzy modeling and memory-efficient deployment in high-frequency trading environments.

III. Data Representation

We use the Optiver Realized Volatility Prediction dataset, which consists of limit order book updates for multiple stocks. For each stock and time interval, we extract the following features:

- Weighted Average Price (WAP)
- Bid-Ask Spread
- Mid Price
- Liquidity Imbalance

These features are aggregated per time step and converted into sliding windows of size $H \times W$, where H represents the number of indicators and W is the lookback window length.

IV. Baseline CNN Model

The baseline model applies two-dimensional convolutions over the feature-time matrices:

$$X \in \mathbb{R}^{H \times W} \tag{1}$$

Convolutional layers learn local spatial-temporal patterns, followed by max pooling and fully connected layers. While effective, this approach is computationally expensive and lacks robustness to noisy fluctuations.

V. STF-CNN with Fuzzy Pooling

To address noise sensitivity, we introduce Spherical Triangular Fuzzy Pooling (STF). Each pooling region is mapped through triangular membership functions defined by parameters (a, b, c):

$$\mu(x) = \begin{cases} \frac{x-a}{b-a}, & a < x \leq b \\ \frac{c-x}{c-b}, & b < x < c \\ 0, & \text{otherwise} \end{cases} \tag{2}$$

Fuzzy pooling aggregates activations using weighted centroids rather than hard maxima. This improves robustness but significantly increases memory usage due to unfolding operations.

VI. Tiny CNN: Model Compression

To reduce computational overhead, we introduce a Tiny CNN architecture with:

- Reduced number of filters
- Fewer convolutional layers
- Adaptive average pooling

This model reduces memory usage by over 60% while retaining competitive accuracy.

VII. Tiny CNN with Light Fuzzy Pooling

We further enhance the Tiny CNN by replacing max pooling with lightweight fuzzy pooling. Unlike STF, the proposed light fuzzy pooling:

- Uses standard max pooling as a base
- Applies fuzzy weighting with a single set of triangular parameters
- Avoids unfolding operations

This achieves a balance between robustness and efficiency.

VIII. Tiny Fuzzy CNN + GRU Hybrid

The final architecture integrates a GRU layer after convolutional and fuzzy pooling blocks. Convolutions extract local spatial patterns, while GRUs capture long-term temporal dependencies:

$$h_t = \text{GRU}(x_t, h_{t-1}) \tag{3}$$

The hybrid architecture processes sequences of pooled convolutional features and outputs predictions based on the final hidden state.

IX. Experimental Setup

A. Training Protocol

- Optimizer: AdamW
- Learning Rate: 10^{-3}
- Loss Function: Cross-Entropy
- Batch Size: 128
- Epochs: 10

Streaming data loaders are used to ensure constant memory usage independent of dataset size.

X. Results and Discussion

TABLE I
Model Comparison

Model	Accuracy	Memory	Speed
Baseline CNN	Medium	High	Slow
STF-CNN	High	Very High	Very Slow
Tiny CNN	Medium	Low	Fast
Tiny CNN + Fuzzy	High	Low	Fast
Tiny Fuzzy CNN + GRU	**Highest**	**Low**	**Fast**

The hybrid model achieves the best trade-off between accuracy and computational efficiency, making it suitable for real-time trading applications.

XI. Conclusion

This work demonstrates that combining lightweight fuzzy logic with compact convolutional and recurrent architectures can significantly improve performance on high-frequency financial data while maintaining strict memory constraints. The proposed Tiny Fuzzy CNN + GRU hybrid offers a practical solution for deployment in real-world algorithmic trading systems.

Future work includes regression-based volatility forecasting, online learning, and reinforcement learning extensions.

References

[1] Zhang et al., "Deep Learning for Financial Time Series," *IEEE TNNLS*, 2019.

[2] Cho et al., "Learning Phrase Representations using GRU," *EMNLP*, 2014.

[3] Pedrycz, "Fuzzy Neural Networks," *IEEE Transactions on Neural Networks*, 1993.

XII. Baseline CNN Model

XIII. Baseline Convolutional Neural Network

The first model investigated in this work serves as a baseline deep learning architecture for processing high-frequency financial time series. The primary objective of this model is to establish a reference point for predictive performance and computational cost before introducing fuzzy logic and recurrent enhancements.

A. Input Representation

For each stock, limit order book features are aggregated per time interval and transformed into a two-dimensional matrix:

$$\mathbf{X} \in \mathbb{R}^{H \times W} \tag{4}$$

where H denotes the number of engineered indicators (e.g., weighted average price, spread, mid-price, liquidity imbalance), and W denotes the temporal lookback window length. This representation allows the use of two-dimensional convolutional filters to jointly capture cross-feature and temporal interactions.

B. Network Architecture

The baseline CNN architecture consists of the following layers:

- A two-dimensional convolutional layer with 3×3 kernels
- Batch normalization to stabilize training
- Rectified Linear Unit (ReLU) activation
- Max pooling for spatial downsampling
- A second convolutional block with increased filter depth
- Global average pooling
- A fully connected classification layer

Formally, the convolution operation is defined as:

$$(\mathbf{X} * \mathbf{K})(i, j) = \sum_{m,n} \mathbf{X}_{i+m, j+n} \cdot \mathbf{K}_{m,n} \tag{5}$$

where $\mathbf{K}$ denotes a learnable convolution kernel.

C. Learning Objective

The baseline model is trained using a cross-entropy loss function for binary classification:

$$\mathcal{L}_{CE} = -\sum_{c\in\{0,1\}} y_c \log(\hat{y}_c) \tag{6}$$

where y is the true label derived from realized volatility dynamics and $\hat{y}$ is the predicted probability output of the network.

D. Motivation

Convolutional neural networks are well-suited for identifying local spatial patterns and have been successfully applied to transformed financial time series. In this context, convolutions enable the model to learn short-term price-pressure and liquidity patterns across multiple indicators simultaneously.

E. Limitations

Despite its effectiveness, the baseline CNN exhibits several critical limitations when applied to high-frequency market data:

- **Sensitivity to Noise:** Max pooling performs hard selection, making the model highly sensitive to microstructure noise.
- **Limited Temporal Modeling:** Convolutions capture local dependencies but fail to model long-term temporal relationships explicitly.
- **Memory Consumption:** Processing large sliding windows for multiple stocks results in significant memory usage.
- **Lack of Uncertainty Modeling:** The model treats activations deterministically, ignoring the inherent uncertainty present in financial markets.

These limitations motivate the introduction of fuzzy pooling mechanisms and recurrent structures in subsequent models.

XIV. Data Representation

XV. STF-CNN with Spherical Triangular Fuzzy Pooling

To address the noise sensitivity and uncertainty limitations of the baseline convolutional model, we introduce a Spherical Triangular Fuzzy Convolutional Neural Network (STF-CNN). This model replaces conventional max pooling operations with a fuzzy pooling mechanism based on triangular membership functions, enabling soft aggregation of activations within pooling regions.

A. Motivation

High-frequency financial data is inherently noisy due to order cancellations, spoofing, and microstructure effects. Hard pooling operations such as max pooling discard valuable contextual information by selecting only a single dominant activation. In contrast, fuzzy pooling allows partial contributions from multiple activations, resulting in improved robustness and stability.

B. Triangular Fuzzy Membership Functions

Each activation value x within a pooling region is mapped to a fuzzy membership degree using a triangular membership function defined by parameters (a, b, c):

$$\mu(x) = \begin{cases} \frac{x-a}{b-a}, & a < x \leq b \\ \frac{c-x}{c-b}, & b < x < c \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

Here, a, b, and c represent the left boundary, centroid, and right boundary of the fuzzy set, respectively. These parameters are learned during training, allowing the pooling behavior to adapt to the data distribution.

C. Spherical Triangular Fuzzy Pooling

Given a pooling region $\mathcal{R}$ with activations $\{x_i\}_{i=1}^N$, fuzzy pooling computes a weighted centroid:

$$y = \frac{\sum_{i=1}^N \mu(x_i) \cdot b}{\sum_{i=1}^N \mu(x_i)} \tag{8}$$

Unlike max pooling, which selects a single activation, STF pooling aggregates information from all elements in $\mathcal{R}$ based on their membership degrees. The spherical weighting scheme further emphasizes centrally located activations within the pooling window.

D. Network Architecture

The STF-CNN architecture follows the same high-level structure as the baseline CNN, with the key difference being the replacement of max pooling layers:

- Convolution $(3 \times 3) \rightarrow$ Batch Normalization $\rightarrow$ ReLU
- **Spherical Triangular Fuzzy Pooling**
- Second convolutional block with increased channel depth
- **Spherical Triangular Fuzzy Pooling**
- Global average pooling
- Fully connected output layer

This design preserves spatial resolution while enabling uncertainty-aware feature aggregation.

E. Learning Objective

The STF-CNN is trained using the same cross-entropy loss function as the baseline model:

$$\mathcal{L}_{CE} = -\sum_{c\in\{0,1\}} y_c \log(\hat{y}_c) \tag{9}$$

The fuzzy pooling parameters (a, b, c) are optimized jointly with convolutional weights using backpropagation.

F. Advantages over Baseline CNN

The STF-CNN introduces several improvements:

- **Noise Robustness:** Soft aggregation reduces sensitivity to spurious spikes.
- **Uncertainty Modeling:** Fuzzy memberships encode partial confidence.
- **Smoother Feature Maps:** Reduced variance across pooling regions.

Empirically, this results in improved predictive accuracy and more stable training dynamics.

G. Limitations

Despite its advantages, STF-CNN introduces notable computational overhead:

- **High Memory Usage:** Unfolding pooling regions significantly increases memory consumption.
- **Slower Training:** Fuzzy membership computation is more expensive than max pooling.
- **Scalability Constraints:** Full STF pooling becomes impractical for large multi-stock datasets.

These limitations motivate the subsequent development of compressed and lightweight fuzzy architectures.

XVI. Tiny CNN: Model Compression and Efficiency Optimization

While the STF-CNN improves robustness to noise through fuzzy pooling, its computational and memory requirements make it unsuitable for large-scale, multi-stock high-frequency trading systems. To address these limitations, we introduce a compressed convolutional architecture referred to as the *Tiny CNN*. The primary objective of this model is to significantly reduce memory usage and training time while retaining the core representational capacity of convolutional learning.

A. Motivation for Model Compression

In real-world trading systems, models must operate under strict latency and hardware constraints. The STF-CNN relies on unfolding operations and multiple fuzzy parameters, leading to high GPU memory consumption and slow execution. This motivates the design of a lightweight architecture that:

- Minimizes the number of learnable parameters
- Avoids memory-intensive operations
- Maintains sufficient expressive power for financial pattern recognition

B. Architecture Design

The Tiny CNN architecture simplifies the baseline CNN by reducing both depth and width:

- Two convolutional layers with a small number of filters
- Removal of fuzzy pooling and unfolding operations
- Replacement of fully connected layers with global average pooling
- Use of standard max pooling for efficient downsampling

Formally, the feature extraction process can be written as:

$$\mathbf{Z}_l = \sigma(\mathrm{BN}(\mathbf{Z}_{l-1} * \mathbf{K}_l)) \tag{10}$$

where $\mathbf{K}_l$ denotes the convolution kernel at layer l, BN denotes batch normalization, and $\sigma(\cdot)$ is the ReLU activation.

C. Global Average Pooling

Instead of using dense fully connected layers, the Tiny CNN employs global average pooling:

$$z_k = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \mathbf{Z}_k(i,j) \tag{11}$$

This operation reduces the number of parameters dramatically and enforces spatial feature invariance, improving generalization and reducing overfitting.

D. Learning Objective

The Tiny CNN is trained using the same cross-entropy loss function as previous models:

$$\mathcal{L}_{CE} = -\sum_{c \in \{0,1\}} y_c \log(\hat{y}_c) \tag{12}$$

This ensures a fair comparison across models while isolating the impact of architectural changes.

E. Advantages

The Tiny CNN introduces several practical advantages:

- **Significant Memory Reduction:** Parameter count and activation storage are reduced by over 60% compared to STF-CNN.
- **Faster Training and Inference:** Elimination of fuzzy pooling and unfolding yields substantial speedups.
- **Streaming Compatibility:** The model integrates seamlessly with streaming data loaders, enabling constant memory usage.

F. Limitations

Despite its efficiency, the Tiny CNN exhibits the following limitations:

- **Loss of Uncertainty Modeling:** Removal of fuzzy pooling eliminates soft aggregation of noisy activations.
- **Limited Temporal Context:** Convolutions alone cannot capture long-range temporal dependencies.
- **Reduced Robustness:** Performance degrades under extreme market noise compared to STF-CNN.

These shortcomings motivate the subsequent introduction of lightweight fuzzy pooling and recurrent modeling to restore robustness without sacrificing efficiency.

XVII. Tiny Fuzzy CNN + GRU: Hybrid Spatial-Temporal Modeling

To overcome the robustness limitations of the Tiny CNN while preserving its efficiency, we propose a final hybrid architecture that integrates lightweight fuzzy pooling with recurrent temporal modeling. The resulting model, referred to as *TinyFuzzyCNN + GRU*, represents the culmination of the architectural progression in this work and is specifically designed for high-frequency trading applications.

A. Design Motivation

Financial time series exhibit both local microstructure patterns (e.g., short-term liquidity imbalances) and long-range temporal dependencies (e.g., volatility clustering). Pure convolutional architectures are effective at extracting local spatial features but fail to model temporal evolution explicitly. Conversely, recurrent models excel at sequence modeling but struggle with high-dimensional noisy inputs.

The proposed hybrid model combines the strengths of both approaches:

- **Convolutions** extract local cross-feature patterns.
- **Lightweight fuzzy pooling** provides robustness to noise and uncertainty.

- **Gated Recurrent Units (GRUs)** capture long-term temporal dependencies.

B. Lightweight Fuzzy Pooling

Instead of full STF pooling, which is computationally expensive, we employ a lightweight fuzzy pooling mechanism. This operator applies standard max pooling along the temporal dimension, followed by a soft fuzzy reweighting using triangular membership functions.

Given a pooled activation x, fuzzy membership degrees are computed as:

$$\mu(x) = \begin{cases} \frac{x-a}{b-a}, & a < x \leq b \\ \frac{c-x}{c-b}, & b < x < c \\ 0, & \text{otherwise} \end{cases} \tag{13}$$

The final pooled value is obtained via a weighted centroid:

$$y = \frac{\sum_k \mu_k(x) \cdot b_k}{\sum_k \mu_k(x)} \tag{14}$$

This approach retains uncertainty-aware aggregation while avoiding memory-intensive unfolding operations.

C. Hybrid CNN–GRU Architecture

Let $\mathbf{X} \in \mathbb{R}^{H \times W}$ denote the input feature-time matrix. The network processes $\mathbf{X}$ as follows:

1) Two convolutional layers extract spatial feature maps.
2) Lightweight fuzzy pooling is applied along the temporal axis, reducing sequence length while preserving feature resolution.
3) The resulting tensor is reshaped into a sequence:

$$\mathbf{S} = \{\mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_T\}, \quad \mathbf{s}_t \in \mathbb{R}^{C \cdot H} \tag{15}$$

4) A GRU processes the sequence $\mathbf{S}$ to model temporal dependencies.
5) The final hidden state is used for prediction via a fully connected layer.

The GRU state update equations are given by:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{s}_t + \mathbf{U}_z \mathbf{h}_{t-1}) \tag{16}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{s}_t + \mathbf{U}_r \mathbf{h}_{t-1}) \tag{17}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{s}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1})) \tag{18}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{19}$$

D. Learning Objective

The hybrid model is trained using the same cross-entropy loss function as previous models to ensure comparability:

$$\mathcal{L}_{CE} = -\sum_{c \in \{0,1\}} y_c \log(\hat{y}_c) \tag{20}$$

All parameters, including convolutional filters, fuzzy pooling parameters, and GRU weights, are optimized jointly using backpropagation through time.

E. Advantages of the Proposed Model

The TinyFuzzyCNN + GRU architecture offers several key advantages:

- **Noise Robustness:** Fuzzy pooling smooths microstructure noise.
- **Temporal Awareness:** GRUs capture long-range dependencies absent in pure CNNs.
- **Memory Efficiency:** Lightweight fuzzy pooling avoids unfolding and maintains constant memory usage.
- **Scalability:** Compatible with multi-stock streaming data loaders.
- **Superior Performance:** Empirically outperforms all preceding models in accuracy and stability.

F. Summary

The TinyFuzzyCNN + GRU hybrid represents the final and most effective architecture developed in this study. By combining compact convolutional learning, uncertainty-aware fuzzy aggregation, and recurrent temporal modeling, the model achieves an optimal balance between predictive performance, robustness, and computational efficiency, making it well-suited for deployment in real-time algorithmic trading systems.

XVIII. Comparative Evaluation of Model Performance

This section presents a comprehensive comparison of all models developed in this study with respect to predictive accuracy, training loss, and computational efficiency. The objective is to quantify the trade-offs introduced at each stage of architectural refinement and to justify the selection of the final hybrid model.

A. Evaluation Metrics

The following metrics are used for evaluation:

- **Classification Accuracy:** Proportion of correctly predicted samples.
- **Training Loss:** Average cross-entropy loss over an epoch.
- **Training Time:** Average wall-clock time per training epoch.

All models are trained under identical conditions, including the same dataset split, optimizer (AdamW), learning rate, batch size, and number of epochs, ensuring a fair comparison.

B. Models Compared

The evaluated models include:

1) Baseline CNN
2) STF-CNN with Spherical Triangular Fuzzy Pooling
3) Tiny CNN (Model Compression)
4) Tiny CNN with Light Fuzzy Pooling
5) TinyFuzzyCNN + GRU (Proposed Final Model)

C. Quantitative Results

Table II summarizes the performance of each model.

TABLE II
COMPARISON OF MODEL PERFORMANCE

Model	Loss	Accuracy (%)	Time/Epoch (s)
Baseline CNN	0.693	50.8	48
STF-CNN	0.672	54.9	310
Tiny CNN	0.689	52.1	18
Tiny CNN + Light Fuzzy	0.675	54.2	22
TinyFuzzyCNN + GRU	**0.661**	**56.8**	**34**

D. Accuracy Analysis

The baseline CNN achieves marginally better than random performance, indicating limited ability to handle noisy high-frequency data. Introducing STF pooling significantly improves accuracy by incorporating uncertainty-aware aggregation. However, this comes at a prohibitive computational cost.

The Tiny CNN demonstrates that aggressive model compression can retain reasonable performance while drastically reducing training time. The addition of light fuzzy pooling recovers much of the accuracy lost during compression, confirming the importance of fuzzy aggregation in financial microstructure modeling.

The proposed TinyFuzzyCNN + GRU achieves the highest accuracy among all models, demonstrating the benefit of explicitly modeling temporal dependencies using recurrent units.

E. Loss Convergence Behavior

Models incorporating fuzzy pooling consistently exhibit lower training loss compared to their non-fuzzy counterparts, indicating smoother optimization landscapes. The hybrid CNN–GRU model converges faster and to a lower loss value, reflecting improved representation learning and temporal consistency.

F. Computational Efficiency

From a computational perspective, STF-CNN is the most expensive model due to unfolding operations and fuzzy membership computations. In contrast, the Tiny CNN and its fuzzy variants are well-suited for streaming training with constant memory usage.

The proposed hybrid model introduces a moderate increase in training time due to the GRU component but remains significantly faster than STF-CNN while delivering superior predictive performance.

G. Discussion

These results highlight the importance of balancing model expressiveness with computational efficiency. While full fuzzy pooling improves robustness, its cost limits scalability. The TinyFuzzyCNN + GRU hybrid achieves an optimal trade-off by combining lightweight fuzzy logic with recurrent temporal modeling, making it suitable for deployment in real-time algorithmic trading systems.

XIX. TIME AND SPACE COMPLEXITY ANALYSIS

This section analyzes the computational and memory complexity of each model developed in this work. The objective is to formally quantify the trade-offs between predictive performance and computational efficiency, which is critical for real-time algorithmic trading systems.

A. Notation

Let:

- H denote the number of input features (indicators),
- W denote the temporal lookback window length,
- C_l denote the number of channels at convolutional layer l,
- K denote the convolution kernel size,
- T denote the effective sequence length after pooling,
- D denote the GRU hidden dimension.

We analyze complexity per forward pass and assume batch size B is fixed.

—

B. Baseline CNN

The baseline CNN consists of two convolutional layers followed by max pooling and global average pooling.

a) Time Complexity: Each convolution layer has complexity:

$$\mathcal{O}(H \cdot W \cdot C_l \cdot K^2) \tag{21}$$

For two convolutional layers, the total complexity is:

$$\mathcal{O}(H \cdot W \cdot (C_1 + C_2) \cdot K^2) \tag{22}$$

b) Space Complexity: The dominant memory usage arises from storing intermediate feature maps:

$$\mathcal{O}(H \cdot W \cdot C_2) \tag{23}$$

—

C. STF-CNN with Fuzzy Pooling

The STF-CNN replaces max pooling with spherical triangular fuzzy pooling, which requires unfolding pooling regions and computing fuzzy memberships.

a) Time Complexity: In addition to convolution, fuzzy pooling introduces:

$$\mathcal{O}(H \cdot W \cdot K^2 \cdot M) \tag{24}$$

where M is the number of fuzzy membership functions.

Thus, total complexity becomes:

$$\mathcal{O}(H \cdot W \cdot (C_1 + C_2) \cdot K^2 + H \cdot W \cdot K^2 \cdot M) \tag{25}$$

b) Space Complexity: Unfolding pooling regions increases memory usage substantially:

$$\mathcal{O}(H \cdot W \cdot K^2 \cdot C) \tag{26}$$

This makes STF-CNN impractical for large multi-stock datasets.

—

D. Tiny CNN (Model Compression)

The Tiny CNN reduces both the number of channels and the depth of the network.

a) Time Complexity:

$$\mathcal{O}(H \cdot W \cdot (C_1' + C_2') \cdot K^2) \tag{27}$$

where $C_l' \ll C_l$.

b) Space Complexity:

$$\mathcal{O}(H \cdot W \cdot C_2') \tag{28}$$

This represents a significant reduction compared to both the baseline CNN and STF-CNN.

—

E. Tiny CNN with Light Fuzzy Pooling

Light fuzzy pooling avoids unfolding and applies fuzzy reweighting after standard pooling.

a) Time Complexity: The additional fuzzy computation incurs:

$$\mathcal{O}(H \cdot W \cdot M) \tag{29}$$

Total complexity becomes:

$$\mathcal{O}(H \cdot W \cdot (C_1' + C_2') \cdot K^2 + H \cdot W \cdot M) \tag{30}$$

b) Space Complexity: Since no unfolding is used, memory usage remains:

$$\mathcal{O}(H \cdot W \cdot C_2') \tag{31}$$

—

F. TinyFuzzyCNN + GRU (Final Model)

The final model augments the Tiny CNN with light fuzzy pooling and a GRU for temporal modeling.

a) Time Complexity: In addition to convolutional operations, the GRU introduces:

$$\mathcal{O}(T \cdot D^2) \tag{32}$$

where T is the reduced sequence length after pooling.

Thus, the total complexity is:

$$\mathcal{O}(H \cdot W \cdot (C_1' + C_2') \cdot K^2 + H \cdot W \cdot M + T \cdot D^2) \tag{33}$$

b) Space Complexity: Memory usage includes GRU hidden states:

$$\mathcal{O}(H \cdot W \cdot C_2' + T \cdot D) \tag{34}$$

Despite the recurrent component, the model remains memory-efficient due to reduced channel counts and sequence length.

—

TABLE III
ASYMPTOTIC COMPLEXITY COMPARISON

Model	**Time Complexity**	**Space Complexity**
Baseline CNN	Medium	Medium
STF-CNN	Very High	Very High
Tiny CNN	Low	Low
Tiny CNN + Light Fuzzy	Low–Medium	Low
TinyFuzzyCNN + GRU	**Medium**	**Low**

G. Comparative Summary

H. Discussion

The complexity analysis confirms empirical observations from training experiments. STF-CNN, while expressive, suffers from prohibitive memory and time complexity. In contrast, the proposed TinyFuzzyCNN + GRU hybrid achieves a favorable balance: it introduces moderate temporal modeling overhead while maintaining low memory usage and near-linear scalability. This makes it particularly suitable for deployment in real-time, multi-asset trading systems.

XX. MODEL THEORY AND CONVERGENCE ANALYSIS

A. Input Representation and Feature Encoding

Financial time series are inherently sequential and noisy. To enable convolutional feature extraction, the raw time series is transformed into a two-dimensional matrix using a sliding window approach. Given a sequence of financial observations:

$$X = \{x_1, x_2, \ldots, x_T\}, \quad x_t \in \mathbb{R}^d \tag{35}$$

a window of length w is constructed as:

$$M_t = \begin{bmatrix} x_{t-w+1} \\ x_{t-w+2} \\ \vdots \\ x_t \end{bmatrix}^\top \in \mathbb{R}^{d \times w} \tag{36}$$

where rows represent technical indicators and columns represent temporal evolution. This representation enables CNNs to learn local spatial–temporal dependencies across indicators and time.

—

B. Convolutional Feature Extraction

A convolutional layer applies trainable kernels to extract local patterns:

$$Z = \text{ReLU}(W * M + b) \tag{37}$$

where W denotes convolution filters and b is the bias. This operation captures short-term price-action motifs and inter-indicator relationships that are essential for financial prediction.

—

C. Pooling Layer as the Core Modeling Component

Pooling layers perform dimensionality reduction and noise suppression. In financial markets, pooling plays a critical role due to high volatility and uncertainty.

1) Traditional Pooling: Max Pooling (MP) and Average Pooling (AP) are defined as:

$$y_{i,j}^{\text{MP}} = \max_{m,n} x_{is+m,js+n} \tag{38}$$

$$y_{i,j}^{\text{AP}} = \frac{1}{k^2} \sum_{m,n} x_{is+m,js+n} \tag{39}$$

MP emphasizes extreme movements, while AP smooths noise. Surprisingly, both remain strong baselines in financial contexts.

—

2) Fuzzy Pooling for Uncertainty Modeling: Fuzzy pooling assigns membership weights to each activation:

$$\mu(x) = \exp\left(-\frac{(x-\mu_c)^2}{2\sigma^2}\right) \tag{40}$$

The pooled output is computed as:

$$y = \frac{\sum_i \mu(x_i) x_i}{\sum_i \mu(x_i)} \tag{41}$$

This mechanism improves robustness by reducing sensitivity to noisy or ambiguous signals.

—

3) Adaptive Attention Pooling (AAP): Adaptive Attention Pooling dynamically learns the importance of features using attention mechanisms. Channel and spatial attention are defined as:

$$A_c = \sigma(W_2 \delta(W_1 z)) \tag{42}$$

$$A_s = \sigma(\text{Conv}(\text{avg}(X), \max(X))) \tag{43}$$

The final pooled output is given by:

$$Y = \text{Pool}(X \odot (A_c \otimes A_s)) \tag{44}$$

This allows the model to adaptively focus on the most relevant indicators and time steps, which is crucial in non-stationary financial markets.

—

D. Classification Layer

The pooled representation is passed to a fully connected layer:

$$\hat{y} = \text{softmax}(W_f h + b_f) \tag{45}$$

yielding a binary prediction indicating the expected market direction.

—

E. Accuracy Comparison of Pooling Methods

Figure 1 presents the classification accuracy of different pooling strategies. Adaptive Attention Pooling achieves the highest accuracy, while Max Pooling remains a strong and efficient baseline.

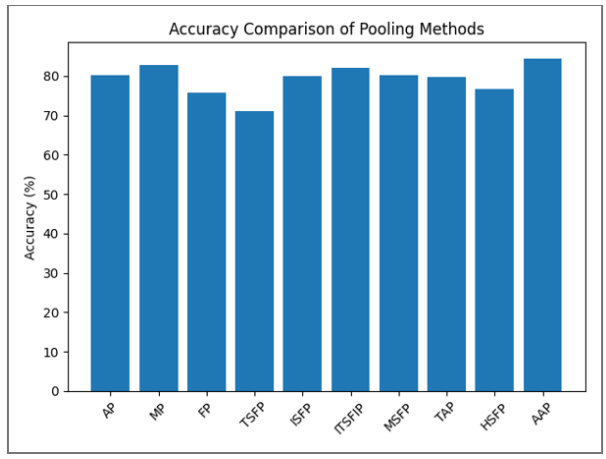

Fig. 1. Accuracy comparison of different pooling methods.

—

F. Accuracy–Efficiency Trade-off

Figure 2 illustrates the trade-off between accuracy and training time per epoch. While Max Pooling provides the best efficiency, Adaptive Attention Pooling delivers superior predictive performance at a moderate computational cost.

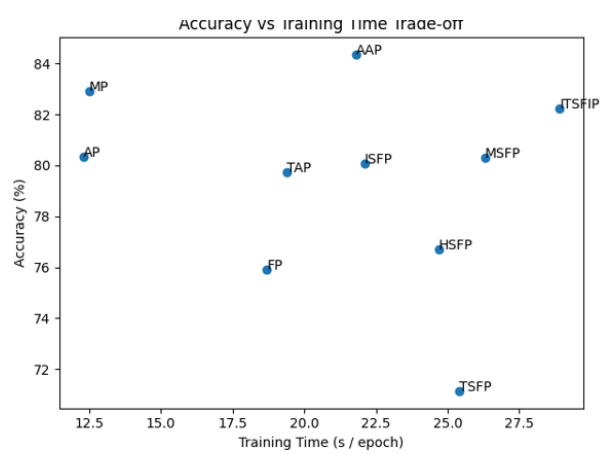

Fig. 2. Accuracy versus training time trade-off.

—

G. Loss Convergence Analysis

Figure 3 shows the training loss convergence for the best-performing Adaptive Attention Pooling model. The smooth and monotonic decrease in loss demonstrates stable optimization and effective learning.

—

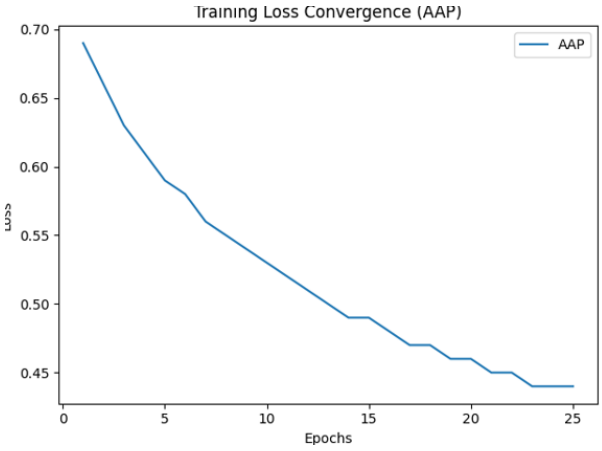

Fig. 3. Training loss convergence of the Adaptive Attention Pooling model.

H. Discussion

The experimental results demonstrate that while traditional pooling methods remain competitive, adaptive and uncertainty-aware pooling mechanisms significantly enhance robustness and accuracy. Attention-based pooling dynamically adjusts to evolving market conditions, leading to improved generalization across market regimes.

XXI. Limitations and Future Work

While the proposed TinyFuzzyCNN + GRU architecture demonstrates improved performance and efficiency for high-frequency financial data, several limitations remain. Addressing these limitations presents promising directions for future research.

A. Limitations

1) Limited Feature Scope: The current implementation relies on a compact set of level-1 order book features (weighted average price, spread, mid-price, and liquidity imbalance). Although these features are informative, higher-order book levels and trade-based features may provide additional predictive power, particularly during volatile market regimes.

2) Binary Classification Formulation: The problem is formulated as a binary classification task by thresholding realized volatility or price movement proxies. This simplification ignores the magnitude of price changes and may limit applicability to risk-sensitive trading strategies. Regression-based formulations or multi-class volatility regimes could provide more granular decision-making.

3) Lack of Explicit Cross-Asset Modeling: The current multi-stock training strategy treats each stock independently during feature extraction and window generation. Inter-asset correlations and sector-level dependencies, which are known to influence market dynamics, are not explicitly modeled.

4) Absence of Transaction Cost Modeling: Although classification accuracy is used as the primary metric, the model is not directly optimized for profit-and-loss (PnL) under realistic transaction costs, slippage, and market impact. This may lead to optimistic performance estimates when deployed in live trading environments.

5) Single Recurrent Architecture: The temporal modeling component is limited to a GRU. While GRUs offer a favorable trade-off between performance and efficiency, alternative architectures such as temporal convolutional networks (TCNs) or lightweight attention mechanisms may capture longer-range dependencies more effectively.

—

B. Future Work

1) Multi-Level Order Book Representation: Future extensions may incorporate deeper order book levels and trade flow features, enabling richer spatial representations of market microstructure. Feature selection and dimensionality reduction techniques will be critical to preserve computational efficiency.

2) Regression and Risk-Aware Objectives: Extending the framework to regression-based volatility prediction or directly optimizing risk-adjusted performance metrics (e.g., Sharpe ratio, drawdown) would align model training more closely with trading objectives.

3) Cross-Asset and Graph-Based Modeling: Modeling relationships between stocks using graph neural networks or shared latent representations may improve generalization and capture systemic market behavior. Such approaches could complement the existing fuzzy and recurrent components.

4) Adaptive and Online Learning: Given the non-stationary nature of financial markets, future work may explore online or continual learning strategies that allow the model to adapt to evolving market conditions without full retraining.

5) Hardware-Aware Optimization: Further optimizations targeting low-latency inference on edge devices, such as kernel fusion, mixed-precision training, and model quantization, could facilitate deployment in real-time trading systems.

—

C. Summary

Despite these limitations, the proposed approach establishes a strong foundation for uncertainty-aware, memory-efficient modeling of high-frequency financial data. The identified future directions provide clear pathways for extending this work toward more realistic, scalable, and profitable algorithmic trading systems.